

StarryOS 大作业总结

支持 jcode 运行的内核调试与修复

学生：姓名

学号：XXXXXXXX

清华大学

2026 年 6 月

目录

- 1 项目背景
- 2 修复概览
- 3 ext4 文件系统
- 4 内存管理与 COW
- 5 epoll 事件通知
- 6 异步任务调度
- 7 网络协议栈

jcode 是什么

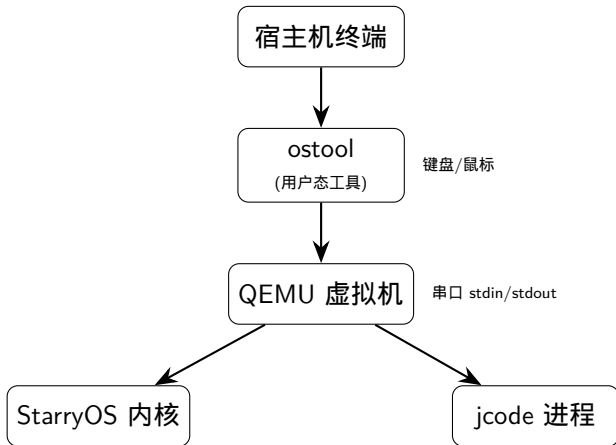
jcode：一个 TUI（终端用户界面）应用

- 架构：TUI 进程 + serve 进程，通过 Unix socket 通信
- 技术栈：ratatui 框架、Tokio 异步运行时、reqwest HTTP 客户端
- 功能：AI 代码助手，支持鼠标点击、窗口调整、网络请求

对内核的要求

- ext4 文件系统（读写文件）
- Unix domain socket（进程间通信）
- epoll + 非阻塞 I/O（Tokio 事件循环）
- TCP/IP 网络（HTTPS 请求）
- TTY 子系统（终端交互、鼠标、窗口大小）

运行架构



修复概览：17 个 bug，7 个内核模块

模块	修复数	典型问题
ext4 文件系统	3	块分配失败、rename 后写入、日志损坏
内存管理	1	COW 缺页 panic
epoll 事件通知	2	ET 竞争窗口、NoEvent 死循环
任务调度	3	device_mask、抢占式唤醒、双重入队
网络协议栈	6	ARP 队列、Unix socket accept/EOF
TTY 终端	3	终端尺寸、SIGWINCH、批量输出
ostool	2	鼠标事件转发、Moved 过滤

Fix 1a：块分配跳过不一致块组

问题

Alpine rootfs 非正常关机后，块组描述符（BGD）声称有空闲块，但位图全满。原始代码遇到 ENOSPC 直接报错 → 整个文件系统报“磁盘已满”。

修复

对 ENOSPC 单独处理，跳过不一致的块组，继续尝试后续块组：

```
let alloc = match alloc_res {
  Ok(a) => a,
  Err(e) if e.code == Errno::ENOSPC => {
    warn!("group={}_inconsistent, skip", group_idx);
    continue; // skip, try next group
  }
  Err(e) => return Err(e),
};
```

Fix 1b : rename 后写入失效

问题

jcode 使用原子替换保存文件 (write tmp → rename tmp → target)。VFS 的 Inode 缓存了 path , rename 后旧路径失效 → Drop 时 sync 用旧路径写 → ENOENT。

修复

```
// old: use path (stale after rename)
rsext4::write_file(dev, fs, &self.path, offset, buf)
// new: use inode number (never changes)
rsext4::write_inode_data(dev, fs, self.ino, offset, buf)
```

根因

Inode 不应该缓存 path。path 是 DirEntry (目录树) 的职责 , 不是 Inode (磁盘实体) 的职责。

Fix 1c : JBD2 日志超级块损坏

问题

Alpine rootfs 的 JBD2 超级块损坏： $s_maxlen=1$ （实际 8192）， $s_start=8192$ （越界）。导致挂载时日志重放失败，文件系统无法挂载。

修复

- $s_maxlen < s_first$ （明显损坏）→ 用物理分区实际长度
- $s_start \geq \text{journal_blocks.len}()$ （越界）→ 视为“日志干净”，跳过重放

Fix 2：内核态 COW 缺页 panic

问题

get_as_mut() 获取用户空间 &mut 引用。另一个线程调用 fork() 将该页标记为 COW 只读。内核写入 → CPU #PF (写保护异常) → panic。

修复（两层）

- ① 主修复：epoll/poll 改用内核缓冲区 + vm_write_slice 安全写回
- ② 兜底：扩展 handle_page_fault，内核态缺页时也能处理用户空间 COW

思考

get_as_mut() 是不安全的访问方式，应该逐步替换为 vm_read/write_slice。

Fix 3a : EPOLLET 竞争窗口

问题

ET 模式要求” 每次状态变化恰好通知一次”。

`mark_not_in_queue()` 和 `register_waker_only()` 之间数据到达 → 通知丢失。

修复

注册新 Waker 后再次检查文件状态。如果数据已在缓冲区，直接将 interest 入队并唤醒 `epoll_wait`。

Fix 3b : NoEvent 路径死循环

TCP socket 的 EPOLLOUT 始终就绪。虚假唤醒 → `check_and_register_waker` → `wake` → `re-queue` → 死循环。修复：NoEvent 路径只注册 Waker，不主动 poll/唤醒。

Fix 4 : device_mask 与 Waker 注册

核心概念

TCP socket 的 device_mask 决定 Waker 注册到哪个网卡设备的 PollSet 上。默认为 0，只有在 start_connect() 内部才被设置。

问题

Tokio 的典型流程：epoll_ctl(ADD) → connect()。epoll_ctl 时 device_mask=0，InterestWaker 注册到空 mask → 等于没注册。connect 设好 mask 后，原来注册的 InterestWaker 不会自动迁移。

修复

poll_io 无论阻塞/非阻塞都先注册 Waker。connect 在 mask 设好后再注册一次。

Fix 5a/5b：抢占式唤醒与双重入队

Fix 5a：抢占式唤醒

原始 `unblock_task(task, false)` 不设置抢占标志，键盘响应延迟 20ms。修复：改为 `unblock_task(task, true)` 立即抢占。

Fix 5b：双重入队

抢占后任务在 `wait_until` 中被唤醒，条件仍不满足（锁未释放），再次入队 → 重复条目。修复：入队前检查 `in_wait_queue` 标志。

为什么条件仍不满足？

`notify_one(resched=true)` 在持有锁的代码中调用。唤醒 T 后锁还没释放。抢占式唤醒让 T 立刻运行，但锁还在别人手里。

ARP 三个修复

Fix 6a : 队列扩容 32 → 128

jcode 启动时并发建立 10-20 个 TCP 连接，32 个槽位耗尽，SYN 包被丢弃。

Fix 6b : TTL 延长 60s → 300s

AI 响应可能超过 60s，ARP 缓存过期后所有 ACK 包重新排队，压爆队列。

Fix 6c : 全队列扫描

原始代码只处理队头。队列 [A, B, A, A] (A 已解析，B 未解析) → 只发第一个 A，后面的 A 永远卡在 B 后面。修复：扫描整个队列，发送所有已解析的包。

Unix socket 两个修复

Fix 6d : 非阻塞 accept

accept() 内部用 rx.recv().await , 完全忽略 O_NONBLOCK。Tokio 用 EPOLLET , 收到 EPOLLIN 后循环 accept 直到 EAGAIN , 但旧代码永远不返回 → Tokio 冻结。修复 : 添加 try_accept() , 无连接时立即返回 WouldBlock。

Fix 6e : EOF 信号顺序

Drop 时 HeapProd 还活着 (Rust 字段 drop 在函数体之后)。对端 poll 看到 write_is_held()=true + 空缓冲区 → 误判为“没关闭”。修复 : 先设 my_tx_closed 标志再 wake。

Fix 7a/7b/7c : 终端子系统三个修复

Fix 7a : 查询真实终端尺寸

ANSI CPR 技术 : `\x1b[9999;9999H` 移动光标到边界 → `\x1b[6n` 请求位置 → 终端回复 `\x1b[rows;colsR`。默认值改为 24x80 (VT100 标准)。

Fix 7b : SIGWINCH 信号

`TIOCSWINSZ` 更新窗口大小后, 检查尺寸变化, 给前台进程组发 `SIGWINCH`。TUI 收到信号 → 重新读取尺寸 → 刷新界面。

Fix 7c : 批量 UART 输出

每个 `\n` 两次写操作 → 锁获取/释放 → 逐行渲染 → 闪烁。修复: 先收集到 `Vec`, 一次写入, 一帧原子发送。

Fix 8a/8b : ostool 鼠标事件

ostool 是什么

宿主机上的用户态工具，不在 QEMU 内。读取终端键盘/鼠标事件 → 写入 QEMU stdin → 读取 QEMU stdout → 显示在宿主机终端。

Fix 8a : 鼠标事件转发

问题：ostool 静默丢弃鼠标事件。修复：编码为 SGR 格式 (\x1b[<Cb;Cx;CyM/m) 转发给 guest。

Fix 8b : 过滤 Moved 事件

每像素一个 Moved 事件 → TUI 每帧重绘 → UART 饱和 → 冻结。修复：丢弃 Moved，只转发点击/滚动/拖拽。

关键收获

- ① **VFS** 分层设计：DirEntry 树（内存缓存）vs Inode（磁盘实体）vs ext4 目录块。Inode 不应该缓存 path。
- ② 异步执行模型：Rust Future = poll 状态机，block_on = 驱动器，poll_io = 通用 I/O 等待。
- ③ **Waker** 注册的生命周期：device_mask 在 connect 时才设置，epoll_ctl(ADD) 可能早于 connect → 注册无效。
- ④ 抢占式调度的副作用：提高响应速度，但引入竞争条件 □ 任务可能在条件还不满足时就被唤醒。
- ⑤ **Rust Drop** 顺序：函数体先执行，字段后 drop → HeapProd 在 wake 时还活着，需要原子标志绕过。

AI 时代 OS 教学思考

传统实验的挑战

AI 能自动完成”实现 syscall”类任务。单模块实验价值下降。

建议方向

- ① 从”实现”转向”调试”：给有 bug 的内核，让学生定位问题
- ② 跨模块系统性问题：涉及多个子系统的交互 bug
- ③ 真实应用驱动：用 jcode 这类真实应用触发边界条件
- ④ 性能调优：从”功能正确”到”性能优化”
- ⑤ 安全性审查：学会审查 AI 生成的代码

Thanks for your attention!

Q & A